

The Design and Analysis of the Symmetric Authenticated Tunneling Protocol

John G. Underhill

Quantum Resistant Cryptographic Solutions Corporation
contact@qrcscorp.ca

March 1, 2025

Contents

1	Introduction	4
1.1	Context and Motivation	4
1.2	Contributions	5
1.3	Organization	5
2	Protocol Architecture	5
2.1	Roles and Trust Assumptions	5
2.2	Master Key	6
2.3	Server Key	6
2.4	Device Key Record	7
2.5	Credential Verifier	7
3	Message Format and State Machines	7
3.1	Packet Header	7
3.2	Message Types	8
3.3	Session-Key Derivation	8
3.4	Client State Machine	8
3.5	Server State Machine	9
3.6	Pseudocode	10
4	Cryptographic Model and Assumptions	10
4.1	Primitive Assumptions	10
4.2	Adversary Capabilities	11
4.3	Freshness and Key Use Assumptions	11
4.4	State and Time Assumptions	12
5	Security Definitions	12
5.1	Server Authentication	12
5.2	Client Authentication	12
5.3	Channel Confidentiality	12
5.4	Ciphertext Integrity	12
5.5	Replay Rejection	13
5.6	Compromise Containment	13
6	Security Analysis	13
6.1	Server Authentication Theorem	13
6.2	Client Authentication Theorem	14
6.3	Channel Confidentiality Theorem	14
6.4	Ciphertext Integrity Theorem	14
6.5	Replay and Ordering Theorem	15
6.6	Compromise Boundaries	15

7	Replay, Freshness, and Failure Behavior	15
7.1	Established-Session Replay	15
7.2	Handshake Replay	15
7.3	Failure Paths	16
7.4	Denial-of-Service Considerations	16
8	Cryptanalysis and Robustness	16
8.1	Domain Separation	16
8.2	Metadata Exposure	16
8.3	Credential Exposure	17
8.4	Branch Compromise	17
8.5	Randomness Failure	17
9	Parameters and Implementation Considerations	17
9.1	Implemented Parameters	17
9.2	SCB Cost Parameters	18
9.3	Serialization and Storage	18
9.4	Implementation Assurance	18
10	Related Work and Positioning	18
10.1	Kerberos	18
10.2	TLS 1.3	18
10.3	AEAD Interfaces	19
10.4	Mobile AKA-Style Systems	19
11	Deployment Guidance	19
11.1	Provisioning	19
11.2	Persistent Leaf Consumption	19
11.3	Credential Handling	19
11.4	Network Exposure	19
12	Limitations and Future Work	20
12.1	No Perfect Forward Secrecy	20
12.2	No Transcript Hash	20
12.3	No Server Nonce	20
12.4	Strict Ordering	20
12.5	Callback Trust Boundary	20
13	Conclusion	20

Abstract

The Symmetric Authenticated Tunneling Protocol (SATP) is a symmetric-only authenticated channel protocol for deployments in which endpoints can be provisioned with device-specific symmetric material before use and in which the operational cost, code footprint, or dependency structure of public-key key establishment is undesirable. SATP uses a deterministic cSHAKE-based hierarchy to derive server and device keys from a master secret, a two-message tunnel establishment exchange to derive bidirectional channel keys, and a post-tunnel client authentication exchange mediated by a server callback. The encrypted transport uses authenticated encryption with associated data, where the serialized SATP header is included as associated data and strict in-order sequence checking is applied before plaintext is accepted. This paper gives a formal description of the implemented SATP construction, states the assumptions under which its security claims hold, and analyzes server authentication, callback-mediated client authentication, channel confidentiality, ciphertext integrity, replay rejection, compromise boundaries, and operational limitations. The analysis intentionally distinguishes between implemented behavior, deployment assumptions, and possible extensions. In particular, SATP does not provide public-key-style perfect forward secrecy: a party that records a handshake and later obtains the corresponding device leaf can recompute that session's keys because the client nonce is transmitted in clear. The protocol instead provides single-use leaf separation, deterministic compromise containment, and efficient post-quantum symmetric security under the secrecy of provisioned keys and the security of SHAKE/cSHAKE, KMAC, SCB, and the selected AEAD primitive.

1 Introduction

1.1 Context and Motivation

Symmetric trust infrastructures remain important in environments where devices are enrolled under a common administrative authority and receive secret material before deployment. Examples include controlled infrastructure networks, embedded access devices, industrial systems, field equipment, and other systems where an operator can provision devices through a trusted channel. In such environments, symmetric cryptography can provide strong post-quantum security margins with low runtime cost, small messages, and compact implementation profiles. These properties are significant where public-key operations, certificate validation, or post-quantum key encapsulation mechanisms are operationally expensive.

SATP is designed for this model. It establishes an encrypted and authenticated duplex channel without asymmetric cryptography, public certificates, or online key distribution servers. A master key derives server or branch keys, server keys derive device key trees, and a client consumes one device leaf for each tunnel establishment attempt. The server reconstructs the active device leaf from the server derivation key and the device identity string supplied by the client. A client-generated nonce is then mixed with the selected device leaf and the protocol configuration string to create transmit and receive channel keys and nonces. The server proves possession of the correct server secret by returning an encrypted validation hash. Client authentication occurs after the encrypted tunnel is raised and is enforced by the server application callback supplied to the SATP server start routine.

The resulting design is not an authenticated key exchange in the same sense as TLS 1.3 or a KEM-based protocol. It does not negotiate algorithms, it does not transmit a server nonce,

and it does not provide perfect forward secrecy against later compromise of the device leaf or branch key. It is instead a deterministic symmetric channel protocol whose security is bounded by the secrecy and lifecycle management of provisioned material. The purpose of this paper is to define that model precisely and analyze the security properties actually provided by the construction.

1.2 Contributions

This paper provides the following contributions.

- It gives an implementation-aligned formal specification of SATP, including the master-to-server derivation, server-to-device derivation, two-message tunnel establishment, post-tunnel authentication exchange, packet header format, and strict sequence processing rules.
- It defines a security model for a symmetric provisioned tunnel protocol, distinguishing server authentication, client authentication, channel confidentiality, ciphertext integrity, replay rejection, and compromise containment.
- It presents proof sketches that reduce the main channel properties to the pseudorandomness of cSHAKE/SHAKE and the confidentiality and integrity of the AEAD primitive, under the assumption that device leaves are not reused and that callback authentication is correctly enforced by the server application.
- It identifies the protocol’s security boundaries. SATP provides single-use leaf separation and deterministic session-key separation; it does not provide perfect forward secrecy against later disclosure of the selected leaf, the server derivation key, or the branch secret.
- It compares SATP with Kerberos, TLS 1.3 in pre-shared-key deployments, and mobile AKA-style symmetric trust models, emphasizing differences in online infrastructure, transcript binding, forward secrecy, and provisioning assumptions.

1.3 Organization

Section 2 describes the SATP architecture and key hierarchy. Section 3 specifies the message formats and state machines. Section 4 defines the cryptographic model and adversary capabilities. Section 5 states the formal security definitions. Section 6 gives the principal security arguments. Section 7 analyzes replay, ordering, and failure behavior. Section 8 discusses compromise boundaries and operational robustness. Section 9 addresses parameter choices and implementation considerations. Section 10 compares SATP with related protocols. Section 11 gives deployment guidance. Section 12 states limitations and future work. Section 13 concludes.

2 Protocol Architecture

2.1 Roles and Trust Assumptions

SATP has two online protocol roles: a client and a server. A provisioning authority exists offline and creates master, server, and device key records. The root or master authority is not part of the online tunnel establishment exchange. The protocol assumes that provisioning is

performed over a trusted or otherwise protected channel, that clients receive device records containing their key tree and the server validation secret, and that the server possesses the server derivation key needed to reconstruct device leaves.

The implemented server application also stores a device credential verifier. During the client authentication phase, the first encrypted message sent by the client contains a device identifier and a 32-byte authentication token. The server application callback uses the device identifier to fetch the stored verifier and calls the SATP passphrase verification interface. The authentication mechanism is therefore part of the supplied server application and the SATP callback interface. It is not hard-wired inside the SATP key-exchange function.

The protocol assumes the following long-term secrets remain protected before compromise events explicitly modeled in the analysis:

- the master derivation key mdk , held by the provisioning authority;
- the server derivation key sdk , held by the server;
- the server validation secret ST_c , held by the server and provisioned to clients in that server branch;
- each unused device leaf $K_{c,i}$, held by the client key-tree record and reconstructable by the server;
- each stored credential verifier used by the server authentication callback.

2.2 Master Key

The master key record is denoted

$$M = (mdk, mid, exp).$$

The field mdk is a 256-bit master derivation key sampled by the random provider. The master identifier mid identifies the administrative root, and exp records the validity period. The master key is an offline provisioning object and is not used during tunnel establishment.

2.3 Server Key

A server key record is denoted

$$S = (sdk, sid, ST_c, exp).$$

The server derivation key is computed by cSHAKE-256 with the master key as the key input, the SATP configuration string as the function-name input, and the server identifier as the customization input:

$$sdk := \text{cSHAKE256}_{\text{key}=mdk, \text{name}=cfg, \text{custom}=sid}(256).$$

The server validation secret ST_c is sampled independently. It is not derived from sdk . The server validation secret is copied into device records and is used to authenticate the server during the connection response.

2.4 Device Key Record

A device key record is denoted

$$D = (\text{KTree}, \text{kid}, \text{ST}_c, \text{exp}, \text{spass}).$$

The reference constant `SATP_KEY_TREE_COUNT` fixes the number of device leaves at 1024. Each leaf has length `SATP_DKEY_SIZE`, which is 32 bytes. The identity kid includes a device identity prefix and a trailing key index. The trailing key-index bytes are incremented in big-endian order during key-tree generation and persistent key-state update.

For each index j , the provisioning algorithm derives a device leaf by cSHAKE-256 with the server derivation key as the key input, no cSHAKE function-name string, and the current kid value as the customization input:

$$K_{c,j} := \text{cSHAKE256}_{\text{key=sdk, name=}\epsilon, \text{ custom=kid}_j}(256).$$

The server reconstructs an active device leaf using the same derivation and the kid value transmitted in the Connect Request. The server does not store client key trees in the SATP runtime model.

2.5 Credential Verifier

Client authentication is callback-mediated. The supplied server application stores a 32-byte credential verifier indexed by the device identifier. The client application sets the spass pointer in the device key record before calling the SATP client connection function. The SATP client then encrypts the pair

$$(\text{did}, H_{\text{pass}})$$

inside the established tunnel and sends it as the first encrypted message after tunnel establishment. The server receive loop routes that first encrypted message to the application callback. The callback fetches the stored credential and verifies it using `satp_server_passphrase_hash_verify`. The security of client authentication therefore depends on the correctness of the supplied callback, the storage protection of the credential database, the entropy of the credential source, and the cost parameters selected for SCB. The SATP library default SCB cost constants are implementation parameters rather than a formal security lower bound.

3 Message Format and State Machines

3.1 Packet Header

Each SATP packet begins with a 21-byte header. The implemented wire order is

$$\text{Header} = \text{flag}[1] \parallel \text{msglen}[4] \parallel \text{sequence}[8] \parallel \text{utctime}[8].$$

The multi-byte fields are serialized in little-endian order. The message length field gives the length of the payload following the header. For encrypted packets, the payload includes ciphertext and authentication tag material as produced by the selected AEAD primitive. The serialized header is passed to the AEAD as associated data before encryption or decryption.

3.2 Message Types

The implemented exchange uses the following online messages.

Message	Direction	Protection	Payload
Connect Request	Client to Server	Plaintext	$\text{kid} \parallel N_h$
Connect Response	Server to Client	AEAD	H_c
Authentication Request	Client to Server	AEAD	$\text{did} \parallel H_{\text{pass}}$
Authentication Response	Server to Client	AEAD	sid
Application Data	Both directions	AEAD	Application payload
Error	Both directions	Context-dependent	Error code

Dedicated authentication flags are defined in the header enumeration, but the implemented authentication request and response are transported as encrypted messages under the SATP channel. Keepalive packet processing is not part of the implemented SATP v1.0a online behavior. Any keepalive identifiers present in the header enumeration are reserved and non-normative for this analysis.

3.3 Session-Key Derivation

The client samples a 32-byte nonce N_h . The session-key expansion is cSHAKE-256 with the active device leaf as key input, the SATP configuration string as function-name input, and the client nonce as customization input:

$$X := \text{cSHAKE256}_{\text{key}=K_{c,i}, \text{name}=cfg, \text{custom}=N_h}(1024).$$

The 128-byte output X is partitioned as

$$X = R_k \parallel R_n \parallel T_k \parallel T_n,$$

where all four fields are 32 bytes. Client and server assign these values to opposite receive/transmit directions. Direction separation is therefore positional rather than label-based. A standards-oriented variant may include explicit role and direction labels, but the reference model analyzed here follows the implemented ordering.

The validation hash is cSHAKE-256 with the client nonce as key input, the selected device leaf as function-name input, and the server validation secret as customization input:

$$H_c := \text{cSHAKE256}_{\text{key}=N_h, \text{name}=K_{c,i}, \text{custom}=ST_c}(256).$$

This construction is a cSHAKE domain-separated computation. It is not modeled as ordinary concatenation SHAKE over $N_h \parallel K_{c,i} \parallel ST_c$.

3.4 Client State Machine

The client state machine is as follows.

1. The client loads a device record and selects the current device leaf. A leaf whose bytes have already been erased is rejected. The client initializes transmit and receive sequence counters to zero.
2. The client samples N_h , computes X , initializes the local AEAD states, computes H_c , and transmits a plaintext Connect Request containing $\text{kid} \parallel N_h$.
3. The client receives a Connect Response. It validates header fields, sets the response header as associated data, decrypts the response, and compares the recovered value to the locally computed H_c .
4. If the validation hash matches, the server is authenticated under the assumption that only the legitimate server can reconstruct $K_{c,i}$ and knows ST_c . The encrypted tunnel is live.
5. The client sends an encrypted authentication request containing $\text{did} \parallel H_{\text{pass}}$. It accepts the authentication response only if the encrypted server identifier matches the expected identifier derived from the device identity layout.
6. After successful authentication, subsequent encrypted messages are application data and must pass sequence, timestamp, and AEAD verification.

3.5 Server State Machine

The server state machine is as follows.

1. The server accepts a socket connection and reads a Connect Request.
2. The server validates the header, extracts kid and N_h , reconstructs $K_{c,i}$ from sdk and kid , computes X , initializes the server receive and transmit AEAD states, computes H_c , and sends H_c encrypted as the Connect Response.
3. The server receive loop treats the first encrypted message after tunnel establishment as the authentication request. The decrypted message is passed to the configured authentication callback.
4. If the callback returns success, the server sends an encrypted authentication response containing sid and accepts later encrypted packets as application data.
5. If authentication fails or a packet fails sequence, timestamp, or AEAD validation, the server terminates the connection state.

The authenticated state is therefore a runtime property enforced by the server receive loop and callback, not by an additional cryptographic transcript message in the key-exchange function.

3.6 Pseudocode

Algorithm 1 SATP client tunnel establishment

Require: Device record (KTree, kid, ST_c , spass)

- 1: Select active leaf $K_{c,i}$ from KTree; reject if erased
 - 2: Sample $N_h \leftarrow \{0, 1\}^{256}$
 - 3: $X \leftarrow \text{cSHAKE256}_{\text{key}=K_{c,i}, \text{name}=cfg, \text{custom}=N_h}(1024)$
 - 4: Split X into (R_k, R_n, T_k, T_n)
 - 5: Initialize local receive/transmit AEAD contexts
 - 6: $H_c \leftarrow \text{cSHAKE256}_{\text{key}=N_h, \text{name}=K_{c,i}, \text{custom}=ST_c}(256)$
 - 7: Send Connect Request (kid, N_h)
 - 8: Receive and decrypt Connect Response using serialized response header as AAD
 - 9: **if** decrypted value $\neq H_c$ **then**
 - 10: abort
 - 11: **end if**
 - 12: Send encrypted Authentication Request (did, H_{pass})
 - 13: Receive encrypted Authentication Response sid
 - 14: accept only if sid matches the expected server identifier
-

Algorithm 2 SATP server tunnel establishment

Require: Server record (sdk, sid, ST_c)

- 1: Receive Connect Request (kid, N_h)
 - 2: Recover $K_{c,i} \leftarrow \text{cSHAKE256}_{\text{key}=sdk, \text{name}=e, \text{custom}=kid}(256)$
 - 3: $X \leftarrow \text{cSHAKE256}_{\text{key}=K_{c,i}, \text{name}=cfg, \text{custom}=N_h}(1024)$
 - 4: Split X into (R_k, R_n, T_k, T_n) and initialize inverse directions
 - 5: $H_c \leftarrow \text{cSHAKE256}_{\text{key}=N_h, \text{name}=K_{c,i}, \text{custom}=ST_c}(256)$
 - 6: Send AEAD-encrypted Connect Response containing H_c
 - 7: Receive first encrypted message and pass plaintext to authentication callback
 - 8: **if** callback rejects **then**
 - 9: abort
 - 10: **else**
 - 11: Send encrypted Authentication Response containing sid
 - 12: **end if**
-

4 Cryptographic Model and Assumptions

4.1 Primitive Assumptions

The analysis models cSHAKE-256 and SHAKE-256 according to the SHA-3 sponge framework standardized by NIST FIPS 202 and SP 800-185. The cSHAKE function is treated as a pseudorandom function keyed by its key input and domain-separated by its function-name and customization inputs. This is an abstraction used for the protocol proof; concrete security depends on the resistance of the Keccak permutation and the absence of implementation defects.

The transport primitive is modeled as an AEAD scheme in the sense of RFC 5116. For each direction, the AEAD is initialized with a 256-bit key and nonce material derived from X . The

associated data is the serialized SATP header. The AEAD is assumed to provide confidentiality under nonce-respecting use and ciphertext integrity against adaptive forgery attempts. For the default RCS configuration, the MAC component relies on KMAC; KMAC itself is specified in SP 800-185. The proof treats RCS as an ideal AEAD under the stated interface rather than proving the internal security of RCS.

SCB is modeled as a memory-hard password hashing or verifier derivation function. Its role in this paper is limited to the client-authentication callback. The theorem for client authentication assumes that the server application correctly stores and fetches verifiers, compares outputs in constant time, and rejects authentication when verification fails. The paper does not claim that any specific default SCB cost parameter provides a universal offline-guessing cost; the cost is an implementation and deployment parameter.

4.2 Adversary Capabilities

The adversary controls the network. It may read, replay, delay, drop, inject, and modify packets. It may open sessions with the server and cause the client or server to process malformed messages. It may obtain transcripts of completed sessions. It may later compromise selected long-term values through explicit compromise oracles. The adversary is computationally bounded but may use quantum search against symmetric keys; therefore 256-bit symmetric keys are interpreted as providing approximately 128-bit brute-force resistance under Grover-style search.

The adversary is not assumed to compromise both endpoints during a session unless a compromise experiment explicitly grants that power. Endpoint compromise, extraction of stored device records, branch key compromise, and credential database compromise are treated separately from the normal network adversary.

4.3 Freshness and Key Use Assumptions

A SATP session is fresh for the main channel security definitions if all of the following conditions hold.

1. The active device leaf $K_{c,i}$ has not been used in another accepted session.
2. The client nonce N_h is sampled uniformly and is not repeated with the same $K_{c,i}$.
3. The device leaf remains secret during the session.
4. The server derivation key sdk and server validation secret ST_c remain secret during the session.
5. The authentication callback correctly rejects invalid credential pairs before application data is accepted.

If these assumptions fail, the corresponding security guarantee is narrowed. In particular, reuse of $K_{c,i}$ with the same N_h recreates the same session tuple (R_k, R_n, T_k, T_n) . Later compromise of $K_{c,i}$ permits recomputation of past session keys from recorded transcripts because N_h is public.

4.4 State and Time Assumptions

Established packets are accepted only when the received sequence number equals the next expected value. The implementation uses strict in-order sequencing, not a sliding replay window. The timestamp field is validated against a bounded time window. The timestamp mechanism provides freshness filtering, but it depends on local clock correctness and should not be interpreted as a substitute for unique keys and nonces.

5 Security Definitions

5.1 Server Authentication

Definition 1 (Server authentication). *A client session satisfies server authentication if, whenever the client accepts a Connect Response, there exists a server holding the corresponding sdk and ST_c that could have computed the accepted value H_c , except with negligible probability.*

This definition is implicit authentication. The client verifies a value encrypted under keys derived from $K_{c,i}$ and equal to a cSHAKE computation involving N_h , $K_{c,i}$, and ST_c . The definition does not claim identity agreement beyond the branch/server identifier encoded in provisioning material.

5.2 Client Authentication

Definition 2 (Callback-mediated client authentication). *A server session satisfies client authentication if application data is delivered to the receive callback only after the first encrypted client message has been accepted by the configured authentication callback, and that callback accepts only credential pairs associated with the presented device identifier.*

This definition explicitly includes the application callback in the trusted computing base. A deployment that supplies a permissive callback or stores weak credential material does not satisfy this definition.

5.3 Channel Confidentiality

Definition 3 (Channel confidentiality). *Let $b \in \{0,1\}$ be a hidden bit. The adversary may choose equal-length plaintexts M_0, M_1 for a fresh, authenticated SATP session. The challenger encrypts M_b under the session's transmit state and returns the ciphertext. The adversary wins if it guesses b . SATP provides channel confidentiality if every efficient adversary has advantage at most negligible in the security parameter plus the advantage of distinguishing the KDF outputs or breaking the AEAD confidentiality experiment.*

The definition applies after the session keys are established and the authentication model for the role under consideration has been satisfied. It does not hide the Connect Request identity or client nonce, both of which are transmitted in clear.

5.4 Ciphertext Integrity

Definition 4 (Ciphertext integrity). *A ciphertext-integrity adversary wins if it causes a fresh SATP session to accept an encrypted packet that was not output by the legitimate peer in the*

correct sequence position, except for exact packets previously delivered in their original position. SATP provides ciphertext integrity if this probability is bounded by the AEAD forgery advantage plus negligible terms from header collision and sequence-state ambiguity.

5.5 Replay Rejection

Definition 5 (Strict replay rejection). *A SATP receiver rejects a replayed established-session packet if the replayed packet has a sequence number different from the receiver’s next expected value or if its authenticated header or ciphertext fails verification. SATP provides strict in-order replay rejection when every accepted established packet increments the receiver sequence state exactly once and no stale packet can be accepted after that increment.*

This is not a reordering-tolerant definition. The reference behavior rejects out-of-order delivery rather than maintaining a sliding window.

5.6 Compromise Containment

Definition 6 (Single-use leaf containment). *A SATP deployment provides single-use leaf containment if compromise of an unused device leaf exposes at most sessions established with that leaf, and if unrelated leaves remain computationally hidden under the cSHAKE derivation from sdk so long as sdk remains secret.*

This is weaker than perfect forward secrecy. It is the relevant compromise notion for a deterministic symmetric key-tree protocol.

6 Security Analysis

6.1 Server Authentication Theorem

Theorem 1. *Assume cSHAKE-256 is a pseudorandom function under the keying and domain-separation inputs used by SATP, the AEAD decryptor releases plaintext only after successful authentication, $K_{c,i}$ is known only to the legitimate client and reconstructable only by the legitimate server, and ST_c is known only to legitimate participants in the branch. Then an adversary that causes a client to accept a Connect Response without a legitimate server computation has advantage bounded by the cSHAKE PRF advantage plus the AEAD forgery advantage.*

Proof. For a client to accept, it must decrypt the Connect Response under the receive state initialized from $X = \text{cSHAKE256}_{K_{c,i}, \text{cf}_g, N_h}(1024)$, and the decrypted plaintext must equal $H_c = \text{cSHAKE256}_{N_h, K_{c,i}, \text{ST}_c}(256)$. If the adversary does not know $K_{c,i}$, it cannot compute the AEAD keys except by distinguishing the cSHAKE output from random or by guessing the resulting key material. If it cannot forge under the AEAD, it cannot cause decryption to release a chosen H_c . If it knows the AEAD keys but not ST_c , it still must produce the validation hash. Replacing cSHAKE with a random function leaves the adversary only a negligible probability of predicting H_c . Therefore an accepting transcript without a legitimate server computation implies either a successful cSHAKE distinguishing event or an AEAD forgery event. $\square$

6.2 Client Authentication Theorem

Theorem 2. *Assume the server application supplies an authentication callback that accepts only if the presented pair $(\text{did}, H_{\text{pass}})$ matches the stored verifier for did , the stored verifier is protected from unauthorized modification, and the first encrypted client message is routed to that callback before application data is delivered. Then no adversary lacking a valid credential pair can cause the server to deliver application data as an authenticated client except with probability bounded by the AEAD forgery advantage and the credential-guessing advantage.*

Proof. The server receive loop decrypts the first encrypted post-handshake packet under the established SATP receive state and passes the resulting plaintext to the configured callback. If the adversary cannot forge the AEAD, it cannot cause the server to process a chosen authentication plaintext without possessing the established channel state. If it possesses the channel state but not a valid credential pair, acceptance requires either guessing H_{pass} for the presented did , corrupting the credential store, or exploiting the callback implementation. These cases are exactly the credential-guessing and implementation-failure terms excluded by the assumptions. Thus callback-mediated client authentication follows from AEAD integrity and the correctness of the callback and credential store. $\square$

6.3 Channel Confidentiality Theorem

Theorem 3. *For a fresh authenticated SATP session, channel ciphertexts are confidential under chosen-plaintext attack up to the confidentiality advantage of the AEAD and the pseudorandomness advantage of the cSHAKE session expansion.*

Proof. The session keys and nonces are derived from $X = \text{cSHAKE256}_{K_{c,i}, \text{cfg}, N_h}(1024)$. Under the freshness assumptions, $K_{c,i}$ has not been used in another accepted session and N_h is unique for the selected leaf. Replacing the cSHAKE expansion by uniform independent values changes the adversary's view only by the PRF distinguishing advantage. With uniform independent key material, the confidentiality of application packets reduces directly to the AEAD confidentiality experiment with the serialized header as associated data. The Connect Request remains visible and is outside the confidentiality claim. $\square$

6.4 Ciphertext Integrity Theorem

Theorem 4. *For a fresh SATP session, an adversary cannot cause acceptance of a new encrypted packet outside the legitimate sequence except with probability bounded by the AEAD ciphertext-integrity advantage.*

Proof. Before decryption, the receiver checks that the packet sequence equals the next expected value and that the packet timestamp is within the permitted window. The serialized header, including flag, length, sequence, and timestamp, is then passed as associated data to the AEAD. Any modification to those header bytes invalidates the authentication tag. Any replayed packet whose sequence is stale fails the strict sequence test. Any new packet at the next expected sequence requires a valid AEAD tag under the current channel state. Acceptance of such a packet without legitimate origin therefore implies a ciphertext-integrity forgery against the AEAD. $\square$

6.5 Replay and Ordering Theorem

Theorem 5. *SATP rejects established-session replay under strict in-order delivery. It does not provide reordering tolerance.*

Proof. The receiver stores the last accepted receive sequence number. A candidate packet must have sequence equal to the stored value plus one. When a legitimate packet is accepted, the stored value is incremented. A replay of any earlier packet has a sequence value less than or equal to the stored value and is rejected before plaintext release. A packet from a later sequence position is also rejected because it does not equal the next expected value. Thus replay is rejected in an in-order stream. The same logic rejects legitimate reordered packets; no sliding window state exists in the analyzed construction. $\square$

6.6 Compromise Boundaries

Lemma 1. *Disclosure of a future unused device leaf does not reveal other device leaves unless the server derivation key is also disclosed, assuming cSHAKE behaves as a PRF.*

Proof. Each device leaf is computed as a cSHAKE output under sdk with a distinct customization value derived from kid. Under PRF security, observing one output does not allow computation of another output for a different customization value without the key sdk. $\square$

Lemma 2. *Disclosure of a device leaf for a recorded session allows recomputation of that session's channel keys.*

Proof. The Connect Request transmits N_h in clear, and cfg is public. The session tuple is computed from $K_{c,i}$, cfg, and N_h . A later adversary that obtains $K_{c,i}$ and has recorded N_h can recompute X . Therefore the construction does not provide perfect forward secrecy with respect to later disclosure of $K_{c,i}$. $\square$

7 Replay, Freshness, and Failure Behavior

7.1 Established-Session Replay

Established encrypted packets receive three layers of rejection before application plaintext is accepted. First, the sequence number must equal the next expected value. Second, the timestamp must fall inside the validity window. Third, the AEAD tag must verify under the serialized header as associated data. The strict sequence check is the principal replay control; the timestamp bounds stale traffic but does not independently prove uniqueness.

This behavior is appropriate for TCP-like ordered streams. It is not appropriate for transports that reorder packets unless an additional ordering layer is introduced. A sliding replay window is not part of the implemented SATP behavior and is not assumed in the proofs.

7.2 Handshake Replay

The Connect Request contains $\text{kid} \parallel N_h$ in clear. The server reconstructs $K_{c,i}$ and derives the session state deterministically from $K_{c,i}$, cfg, and N_h . Therefore a replayed Connect Request can cause the server to recreate the same channel state if the timestamp and sequence validation

accept the request and if the server does not maintain consumed-index state. The reference construction relies principally on client-side single-use leaf consumption and nonce freshness. A deployment that requires server-side replay rejection for Connect Requests should maintain a consumed-index database or mix a server nonce into the session derivation. Those mechanisms are outside the analyzed reference behavior.

7.3 Failure Paths

Authentication failure, sequence failure, timestamp failure, malformed header failure, and AEAD verification failure all terminate the affected connection path. Sensitive temporary buffers used in key exchange and authentication are erased by the implementation where they are held in explicit temporary state. Long-term serialized provisioning records require external storage protection and secure lifecycle management.

7.4 Denial-of-Service Considerations

SATP is computationally inexpensive before authentication. This is beneficial for constrained devices but creates an availability consideration for servers exposed to untrusted networks. A server may allocate connection state and receive buffers before client authentication completes. Implementations should apply socket timeouts, connection limits, rate limits, and conservative allocation policies. The server authentication callback should avoid expensive credential verification until the encrypted tunnel has been established and the message structure has been validated.

8 Cryptanalysis and Robustness

8.1 Domain Separation

The construction uses cSHAKE domain separation in several places. Server derivation uses the SATP configuration string as the cSHAKE function-name input. Device-leaf derivation uses no function-name input and places kid in the customization input. Session derivation uses $K_{c,i}$ as key, cfg as function name, and N_h as customization. The validation hash uses N_h as key, $K_{c,i}$ as function name, and ST_c as customization. These input positions are part of the implemented construction and must be modeled as cSHAKE framing rather than ordinary concatenation.

A future variant may strengthen domain separation by assigning explicit labels such as `satp.server`, `satp.device`, `satp.client-tx`, and `satp.server-tx`. The analyzed reference implementation uses the domain separation described above.

8.2 Metadata Exposure

The Connect Request is not encrypted. A passive observer learns the device key identity string and client nonce. This is a deliberate tradeoff in the two-message design. All later authentication and application payloads are encrypted and authenticated, but packet length, timing, and connection metadata remain visible at the transport layer.

8.3 Credential Exposure

The client sends the stored authentication token over the encrypted SATP channel. If the device record is compromised, the adversary may obtain unused device leaves, ST_c , and the client authentication token. This is endpoint compromise and is outside ordinary network security. Deployments should store device records in secure storage, rotate device credentials after suspected extraction, and treat loss of ST_c as a branch-level event.

8.4 Branch Compromise

Disclosure of sd_k permits derivation of all device leaves for identities in the branch. If ST_c is also disclosed, an adversary can impersonate the branch server to devices that trust that secret. Because N_h is visible in recorded Connect Requests, disclosure of sd_k permits recovery of past session keys for recorded sessions whose device identities are known. Epoch rotation and branch revocation are therefore necessary operational controls.

8.5 Randomness Failure

If a client reuses N_h with the same $K_{c,i}$, the session tuple is repeated. If the device leaf is consumed exactly once, nonce reuse with the same leaf cannot occur through ordinary operation. The implementation must reject random-generation failure and must persist leaf consumption before or during connection establishment in a manner resilient to power loss. The supplied client application updates the serialized device key state before initiating the network connection; deployments should preserve that ordering.

9 Parameters and Implementation Considerations

9.1 Implemented Parameters

Parameter	Value	Meaning
Device leaf size	32 bytes	cSHAKE-derived device key
Server validation secret	32 bytes	Secret shared between server branch and devices
Client nonce	32 bytes	Client-generated session nonce
Session expansion	128 bytes	Split into four 32-byte values
Key-tree count	1024	Number of leaves in the device record
SATP header size	21 bytes	Flag, length, sequence, timestamp
Sequence rule	strict $rxseq + 1$	In-order delivery only
Authentication token size	32 bytes	Device credential token passed to callback

9.2 SCB Cost Parameters

The SATP code exposes SCB cost constants used by the server passphrase hashing and verification helpers. These constants are implementation defaults. Security analysis of password-derived credentials must use the actual deployment parameters, the entropy of the underlying passphrase or token, and the protection of the credential store. This paper therefore does not assert a fixed universal offline-guessing cost for all SATP deployments.

9.3 Serialization and Storage

SATP serializes master, server, and device key records into fixed-size binary records. The records contain sensitive material and must be protected by the deployment environment. The base SATP serialization format is not itself an encrypted storage container and should not be treated as a complete secure-storage system. Device records should be stored in tamper-resistant memory, secure elements, encrypted local storage, or an equivalent mechanism appropriate to the deployment risk.

9.4 Implementation Assurance

The principal implementation assurance requirements are deterministic bounds checking, strict input validation, secure erasure of temporary secrets, persistent device-key state update, and complete negative testing of packet parsing. Tests should include known-answer derivation vectors, replayed Connect Requests, replayed established packets, wrong credential pairs, stale timestamps, sequence gaps, malformed lengths, altered header AAD, and corrupted authentication tags.

10 Related Work and Positioning

10.1 Kerberos

Kerberos V5 is a symmetric-key network authentication protocol standardized in RFC 4120. Kerberos relies on an online key distribution center that issues tickets and session keys. SATP does not use a ticket-granting service during tunnel establishment. Instead, it relies on prior provisioning and deterministic server reconstruction of device leaves. This reduces online infrastructure but transfers more responsibility to provisioning, credential storage, and device key-state management.

10.2 TLS 1.3

TLS 1.3, specified in RFC 8446, provides an authenticated transport security protocol with transcript binding, negotiation, and support for ephemeral Diffie-Hellman key exchange and pre-shared-key modes. In ordinary ephemeral configurations, TLS 1.3 provides forward secrecy that SATP does not claim. SATP removes public-key operations, certificate processing, and negotiation from the online path. The cost is narrower security: compromise of SATP branch material or the selected device leaf can affect recorded sessions.

10.3 AEAD Interfaces

RFC 5116 defines an interface for authenticated encryption with associated data. SATP follows the AEAD model at the packet layer by authenticating serialized header bytes as associated data. The design follows the general principle that unencrypted routing or framing fields must still be integrity-protected whenever they affect decryption or state transitions.

10.4 Mobile AKA-Style Systems

Mobile authentication systems also rely on pre-provisioned symmetric secrets and challenge-response procedures. SATP resembles these systems in its use of provisioned device material, but differs in that the result of authentication directly initializes an application tunnel rather than merely authenticating access to a cellular network. SATP also places replay and ordering checks into the tunnel packet layer.

11 Deployment Guidance

11.1 Provisioning

Provisioning must ensure that device identifiers are unique within the server branch, that key-tree records are generated with the correct server derivation key, and that the client receives the correct ST_c value for the server branch. The provisioning process should also initialize credential records atomically so that a device key and its server-side credential cannot diverge.

11.2 Persistent Leaf Consumption

Single-use leaf behavior depends on persistence. A client must update the serialized device key state so that a consumed leaf cannot be reused after restart or power loss. If the persistent update fails, the connection attempt should be aborted. A server-side consumed-index database may be added by deployments requiring independent rejection of replayed Connect Requests.

11.3 Credential Handling

The server authentication callback is security critical. It must validate message length, fetch the credential corresponding to the supplied device identifier, perform constant-time comparison through the SATP verification helper, and reject when lookup fails. The callback should not treat missing records as zero-valued credentials. Credential buffers should be erased after use.

11.4 Network Exposure

Internet-facing deployments should apply admission controls, timeouts, and connection limits. The protocol is more naturally suited to controlled access networks, embedded systems, private infrastructure, and environments where endpoint enrollment is administered centrally.

12 Limitations and Future Work

12.1 No Perfect Forward Secrecy

SATP does not provide perfect forward secrecy. A recorded Connect Request contains N_h , and the session tuple is deterministic from $K_{c,i}$, cfg , and N_h . Later disclosure of $K_{c,i}$ or sdk can therefore allow reconstruction of past sessions. The appropriate claim is single-use leaf containment and deterministic session separation, not public-key-style forward secrecy.

12.2 No Transcript Hash

The reference key schedule does not include a transcript hash over the handshake bytes. Server authentication is provided through H_c , and packet headers are authenticated as AEAD associated data for encrypted packets. A future standards-oriented version could include a transcript hash in both the session expansion and confirmation value.

12.3 No Server Nonce

The server does not contribute fresh nonce material to the session derivation. The session freshness model depends on the client nonce and single-use leaf behavior. Adding a server nonce would reduce replay-state duplication risk and make the session tuple depend on both endpoints.

12.4 Strict Ordering

SATP rejects out-of-order packets. This simplifies replay logic and is suitable for reliable in-order streams, but it does not tolerate reordering. A datagram or lossy transport variant would require a different replay window and AEAD state model.

12.5 Callback Trust Boundary

Client authentication is only as strong as the configured callback and credential store. A permissive callback, a weak credential, or a failed lookup that is not treated as rejection invalidates the client-authentication theorem. Future work may define a fully normative in-protocol credential record and verification procedure.

13 Conclusion

SATP is a compact symmetric tunnel protocol for controlled environments with pre-provisioned device secrets. It uses cSHAKE-based deterministic key hierarchy, a two-message tunnel establishment exchange, authenticated encryption with serialized header associated data, strict sequence processing, and post-tunnel callback-mediated client authentication. Under the assumptions stated in this paper, SATP provides server authentication, callback-mediated client authentication, channel confidentiality, ciphertext integrity, and strict in-order replay rejection.

The protocol's guarantees are intentionally scoped. SATP does not provide perfect forward secrecy, transcript-bound authenticated key exchange, reordering-tolerant replay windows, or

anonymity. Its principal value is efficient symmetric security in systems where provisioning is already part of the deployment model and where the cost or complexity of public-key mechanisms is undesirable. Correct deployment requires strong protection of device records, reliable leaf consumption, credential-store integrity, callback correctness, and operational rate limiting. Within those constraints, SATP provides a clear and analyzable symmetric alternative for post-quantum-oriented secure channels.

References

- [1] QRCS Corporation. *Symmetric Authenticated Tunneling Protocol (SATP) Technical Specification*. Quantum Resistant Cryptographic Solutions Corporation, 2025. Available at: https://www.qrcscorp.ca/documents/satp_specification.pdf.
- [2] QRCS Corporation. *SATP Source Code Repository*. Quantum Resistant Cryptographic Solutions Corporation. Available at: <https://github.com/QRCS-CORP/SATP>.
- [3] National Institute of Standards and Technology. *FIPS 202: SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. U.S. Department of Commerce, 2015. DOI: <https://doi.org/10.6028/NIST.FIPS.202>.
- [4] Kelsey, J., Chang, S., and Perlner, R. *NIST SP 800-185: SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash*. National Institute of Standards and Technology, 2016. DOI: <https://doi.org/10.6028/NIST.SP.800-185>.
- [5] McGrew, D. *RFC 5116: An Interface and Algorithms for Authenticated Encryption*. Internet Engineering Task Force, 2008. Available at: <https://www.rfc-editor.org/rfc/rfc5116>.
- [6] Bellare, M., and Namprempre, C. *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*. Cryptology ePrint Archive, Report 2000/025, 2000. Available at: <https://eprint.iacr.org/2000/025>.
- [7] Rescorla, E. *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*. Internet Engineering Task Force, 2018. Available at: <https://www.rfc-editor.org/rfc/rfc8446>.
- [8] Neuman, C., Yu, T., Hartman, S., and Raeburn, K. *RFC 4120: The Kerberos Network Authentication Service (V5)*. Internet Engineering Task Force, 2005. Available at: <https://www.rfc-editor.org/rfc/rfc4120>.